

ATANDA Command Centre — ATLAS PromptWare Design Document

Investor Edition · v2.0 · Codebase-of-record snapshot

Produced by applying the F6 ATLAS Drafter shape (CheatSheet · ExecSummary · Worksheet · Implementation) to the entire codebase as of the latest merged checkpoint. This document supersedes the v1 PWDD issued at Final Checkpoint 1 and reflects everything shipped since: LLM-agnostic provider selection, per-engine model overrides, the Advanced Cartridge Ingestion premium SKU, the two new Senior badges (AISA_PWDD + AISE_BUILD), shareable badge certificates, admin badge revocation, per-artifact provider/model attribution, and the historical provider backfill.

PART 1 — CHEAT SHEET

The 60-second pitch. If you read nothing else, read this.

Field	Value
What we sell	A subscription portal that runs a vague product idea through an 8-stage AI prompt-engineering pipeline and produces a certified, deploy-ready codebase at the end.
Who buys it	Founders, PMs, and engineering leaders who already use ChatGPT / Claude / Cursor but can't get *consistent, audit-grade* output.
Why it works	We sell the **pipeline**, not the chatbox. Each stage is contract-first, rate-limited, gated by tier, certified at the end, and now **LLM-agnostic** — pick Claude, OpenAI, or Gemini per session or per engine.
What's new since v1 PWDD	Provider dropdown (Claude / OpenAI / Gemini), per-engine model override, Advanced Cartridge Ingestion (\$499.99 / project), two new Senior badges, shareable badge certificates (PNG / JPEG / SVG), admin badge revocation with structured audit log, per-artifact provider+model attribution, retroactive backfill.
Revenue shape	Recurring subscription (4 tiers, monthly + yearly) + one-time per-project ingestion credit + one-time **\$499.99 Advanced Cartridge** premium SKU + Institution / enterprise tier.
Defensibility	Eight ordered engines, certificate verification surface, persistent skill graph (10 badges), and now a premium on-ramp that consumes a customer's entire knowledge library + existing prompt assets + codebase/database link.

Operational posture	Idempotent Stripe webhook, external-first account deletion, SSRF guard on every operator-supplied URL, strict JIT auth sync, Sentry-instrumented on both API and web, per-call LLM telemetry now including provider.
Deployment	One click on Replit. TLS handled. Custom domains supported. No customer-supplied AI keys anywhere in the stack.
Codebase status	Contract-first (OpenAPI single source of truth), typechecks clean across every package, all in-flight tasks listed in the appendix.

PART 2 — EXECUTIVE SUMMARY

The 5-minute pitch. Show this to a partner before a meeting.

2.1 What changed since v1

The v1 PWDD shipped a complete F1 !' F8 HARNESS, Stripe billing, ingestion engine, ten-badge quest system, and demo mode. The codebase is now materially larger and substantially more defensible. Three strategic shifts landed:

1. **LLM-agnostic.** Every engine call resolves a provider per request — Claude (default), OpenAI, or Gemini — through the Replit AI Integrations proxy. We never touch customer keys. Provider is stored per session, overridable per engine call, and persisted on every artifact so an operator can prove (and a buyer can audit) which model produced which design document. Telemetry distinguishes provider and model id for per-engine cost reconciliation.
2. **Advanced Cartridge Ingestion** — a \$499.99 one-time premium SKU positioned above the entry-level ingestion credit. Begins with a mandatory **Define Project Scope** step ("In one sentence, what should this project achieve?"). Then accepts a multi-document Body of Knowledge library, one or more SPCs, and an optional Git / database link. Every subsequent engine call receives a deterministic CARTRIDGE CONTEXT block whose first line is the project scope — protected from truncation — so no downstream stage can drift off the operator's purpose.
3. **Senior reputational tier on the badge wall.** Two new badges (Senior Architect, Senior Engineer) sit above the existing ASPE/AISA/AISE trio with distinct gold styling. Architect auto-awards at 3 certified PWDDs; Engineer requires a verified live URL pointing at an actual AI agent or app built from one of the operator's PWDDs. Both are downloadable as PNG / JPEG / SVG certificates; both are revocable by admins with a structured audit log.

2.2 What the product does, end to end

A signed-in operator opens a session and runs F1 !' F8 in sequence, with optional side-step engines (F6-VDJ for IDE recommendation, DE-SPC for SPC evolution) and an optional pre-pipeline ingestion step. Each engine has a hardened v2 system prompt, a typed input/output contract validated by generated Zod, a tier gate, a daily rate-limit counter, and best-effort telemetry. The terminal output is either:

- a **SPARTAN-certified MVP PDD** (manual session), or
- a **PromptWare Design Document (PWDD)** (ingested session — single doc via the entry-level

ingestion engine, or multi-document via the new Advanced Cartridge),

and, for Architect-tier operators, an optional **F8 Code DJ** scaffold that consumes the certified design document and emits up to 12 files plus a manifest, refusing any input that lacks a SPARTAN cert.

2.3 Why investors should care

- The product is finished, not aspirational. Every stage ships end-to-end with payments, auth, and observability.
- The pipeline is the moat. Reproducing eight ordered, contract-first, certified engines is not a sprint for a chatbox competitor.
- Revenue is mixed and defensible: low-friction one-time credits land non-subscribers, recurring subscriptions retain power users, the \$499.99 cartridge SKU pulls in serious teams with existing knowledge bases, and an Institution tier opens the enterprise door.
- LLM-agnostic positioning eliminates the "you must use Claude" objection and gives us per-provider unit economics from day one.
- A skill graph (10 badges, two now Senior) creates retention pressure and provides downstream verification surface that links back to the product (operators paste cert / badge URLs in pitch decks and portfolios).
- Engineering posture is mature: idempotent webhooks, external-first deletions, SSRF guards everywhere a URL is accepted, strict JIT auth sync, structured audit logs for admin actions, telemetry as best- effort. These are the marks of a team that has shipped before.

2.4 Business model snapshot

SKU	Price	Buyer profile	What it unlocks
Explorer	Free	Trial / casual	F1–F4 with strict daily caps; Claude only
Practitioner	Subscription	Solo operators	F5/F6/F7 with raised caps; all LLM providers; DE-SPC with ASPE badge
Architect	Subscription	Senior operators	Everything unlimited + F8 Code DJ (2 / day)
Institution	Enterprise	Teams	Everything unlimited including F8
Ingestion Credit	One-time per project	Anyone	Single-doc ingestion !' PWDD seed
Advanced Cartridge	**\$499.99 one-time per project**	**Serious teams**	**Multi-doc + SPC + Git/ DB link + scope- anchored pipeline**

2.5 The risks that matter, and the mitigations

Risk	Mitigation in the codebase
Hallucination in certified output	Every engine validates output through a Zod schema before persisting; F8 refuses any input without a SPARTAN cert; CARTRIDGE CONTEXT is scope- anchored and length-bounded.

LLM vendor concentration	LLM-agnostic provider abstraction in <code>`engines/shared.ts`</code> . Switching providers is a per-call argument.
Stripe webhook double-counting	Every event id is recorded in <code>`stripe_webhook_events`</code> (PK on <code>event_id</code>); replays return idempotently; handler errors roll back the idempotency row so Stripe can legitimately retry.
SSRF via operator-supplied URLs	One canonical <code>`headOk`</code> helper resolves the hostname and refuses RFC1918, loopback, link-local, cloud metadata, CGNAT, and multicast; HTTPS only. Reused by the new Senior Engineer badge claim.
Orphaned Stripe subs on account delete	External-first deletion order: Stripe !' Clerk !' local row !' goodbye email. Documented and load-bearing.
Badge fraud	Senior Engineer badge is revocable via <code>`POST /admin/badges/revoke`</code> with structured audit log (event=badge.revoke, target user, badge id, reason, row id, timestamp).
Per-engine cost blow-up	<code>`harness_engine_runs`</code> records provider, modelId, tokens in/out, cost (USD), and duration on every call. Best-effort writes — telemetry failures never break user requests.
Cartridge consuming credit on a failed run	Atomic <code>`FOR UPDATE SKIP LOCKED`</code> claim with release-on-failure mirrors the proven ingestion-credit pattern.

PART 3 — WORKSHEET

The architecture diligence sheet. The shape of the code, the contracts, and the guarantees.

3.1 Stack of record

Layer	Choice	Why it survived diligence
Runtime	Node.js 24, TypeScript 5.9	Modern async, strict types end-to-end
Repo	pnpm workspaces	Deterministic, monorepo-native, fast install
API	Express 5	Mature middleware ecosystem, raw-body Stripe support
Auth	Clerk (Replit-managed tenant)	OAuth + magic link, zero secret handling on our side
Billing	Stripe	Industry standard; idempotent webhook; metadata-keyed flows
Database	PostgreSQL + Drizzle ORM	Type-safe queries; one file per table; no RLS net (app-layer auth only)

Validation	Zod v4 + `drizzle-zod`	Runtime types matched to the DB shape
API codegen	Orval (from OpenAPI)	One contract != server Zod + client React Query hooks
LLM	Anthropic Claude Sonnet 4 + OpenAI + Gemini, all via Replit AI Integrations	Provider-agnostic, no customer keys
Front-end	React 19 + Vite + Wouter + TanStack Query	Fast HMR, lightweight router, mature data layer
UI	Tailwind + shadcn/ui	Designer-trustable, component reuse
Observability	pino logs, Sentry (API + web), `harness_engine_runs` telemetry	Logs, errors, AI cost — all visible
Email	Resend (console-log fallback in dev)	Transactional, modern
Deployment	Replit	One-click deploy, TLS, custom domains
PDFs	pdfkit + marked	Investor-grade docs straight from markdown

3.2 The HARNESS pipeline — current and complete

Stage	Role	Tier gate	Rate cap (Architect)	Streams?
F1	Prompt Diagnostic (JCSE rubric)	All	unlimited	No
F2	Atomic Prompt + Context Craft pillar awards	All	unlimited	No
F3	Micro Agent Birth Package	All	unlimited	**Yes (SSE)**
F4	Micro PDD	All	unlimited	No
F5	Full SPC (interactive)	Practitioner+	unlimited	No
F6	4-Part ATLAS PDD	Practitioner+	unlimited	No
F6-VDJ	IDE / coding-vibe recommender (side-step)	Practitioner+	unlimited	No
F7	SPARTAN Compressor != certified MVP PDD / PWDD	Practitioner+	unlimited	**Yes (SSE)**
F8	Code DJ — scaffold codebase from certified PDD	Architect only	2 / day	No

DE-SPC	Digital Evolution — evolve an existing SPC	Practitioner+ with ASPE badge	per tier rules	No
--------	--	-------------------------------	----------------	----

Every engine now also resolves an **LLM provider** at the start of its **provider > call resolution order** **provider > Claude** default. Explorer requests carrying a **non-Claude** provider receive a typed 403 (**ProviderRequiresTierError**). Missing or unconfigured provider integrations produce a typed 503 (**ProviderNotConfiguredError**). Every successful call records provider, modelId, input tokens, output tokens, cost (USD), and duration.

3.3 The Advanced Cartridge — what actually happens on the wire

```

[ Operator clicks "Start Cartridge Ingestion" ]
    %
    %%%
[ Wizard Step 1: DEFINE PROJECT SCOPE !• required, pinned, validated ]
    { projectName, outcomeOneLiner, scopeStatement, targetPlatformHint? }
    %
    %%%
[ Wizard Step 2: ASSETS ]
    - up to N documents (.txt / .md / .pdf / .docx)
    - up to N SPCs (paste or upload)
    - optional Git URL or database descriptor
    %
    %%%
[ Wizard Step 3: REVIEW ]
    scope shown verbatim at the top so the operator confirms intent
    %
    %%% POST /api/cartridge (multipart)
[ Server: validate scope !' typed 400 SCOPE_REQUIRED if absent ]
    %
    %%%
[ Atomically claim 1 cartridge_credit (FOR UPDATE SKIP LOCKED) ]
    - released on any failure path
    - linked to the cartridge_package on success
    %
    %%%
[ Extract text per document (pdf-parse / mammoth) ]
[ Normalise each via callLlmJson (provider-aware) !' per-doc summary ]
[ Rewrite scope statement !' F1-ready seed prompt ]
    %
    %%%
[ Persist cartridge_packages + cartridge_documents + cartridge_spcs
  + cartridge_links ]
    %
    %%% POST /api/cartridge/:id/start-session
[ Create harness_sessions row with origin='cartridge' + cartridgeId ]
[ Unlock F1-F7 as AVAILABLE so the operator can drive the pipeline ]
    %
    %%%
[ Every subsequent engine call:
  callLlm !' cartridge-context.ts loads the cartridge, builds the
  CARTRIDGE CONTEXT block (scope first, protected from truncation;
  24 KB cap; 60 s cache), prepends to the engine system prompt ]
    %
    %%%
[ F1 !' F7 !' SPARTAN-certified MVP PDD / PWDD ]
    %
    %%% (optional, Architect-tier)
[ F8 Code DJ !' CODEBASE_BUNDLE ("d12 files + manifest) ]

```

The scope statement is the **first thing every engine sees** for the life of the session. It cannot be truncated away. This is what makes the cartridge premium: it buys the operator a focused, purpose-anchored pipeline run, not a generic ingestion.

3.4 LLM provider abstraction at a glance

[illegible]

Lazy client init for OpenAI and Gemini means a missing integration cannot crash startup; calling code receives a typed `ProviderNotConfiguredError` at request time.

3.5 Badge wall — all ten badges

Badge	Type	Earned by	Acronym storage id	Notes
ASPE — Atomic Super Prompt Engineer	Live	"e3 SPCs + "e4 MAS	`ASPE`	Gates DE-SPC
AISA — Atomic Intelligent Systems Architect	Live	"e1 ATLAS + "e1 Micro PDD + "e1 MVP PDD	`AISA`	Composite craft
AISE — Atomic Intelligent Systems Engineer	Stored	Verified live URL on a certified MVP PDD	`AISE`	Evidence-trust badge; revocable
Senior Architect — Advanced Intelligence Systems Architect	Live	"e3 certified PWDDs / MVP PDDs	`AISA_PWDD`	New top-tier; gold styling; downloadable cert
Senior Engineer — Advanced Intelligent Systems Engineer	Stored	Operator-submitted live URL of an AI agent / app built from a PWDD, passed through SSRF guard, plus 500-char evidence note	`AISE_BUILD`	New top-tier; revocable by admin; downloadable cert
Context Craft × 7	Stored	Per-pillar JCSE sub-score "e 6 in any F1/ F2 artifact	`(pillar)`	S / R / I / D / F / E / C

Storage ids deliberately disambiguated to avoid colliding with the legacy trio. Display names use the full prose titles. Senior badges render above the existing wall with a gold gradient + hex-clip tile and a PNG / JPEG / SVG download menu.

3.6 Admin badge revocation, appeal, and restore flow

The evidence-trust badges (AISE and AISE_BUILD) now have a full lifecycle: claim !' revoke !' user appeal !' admin restore. None of it is silent.

Revoke — `POST /api/admin/badges/revoke` (`requireAuth` + `requireAdmin`):

1. Validate body: `{ userId, badgeId in [AISE, AISE_BUILD], reason 1..1000 }`.
2. Confirm target user exists (404 otherwise).
3. **Soft-delete** the `command_centre_badges` row: set `status='REVOKED'`, stamp `revokedAt`, `revokedReason`, `revokedByUserId`, and append the full record to a `revoke_history` jsonb array (repeat revocations are visible on the row itself).
4. Insert a row into the dedicated `badge_revocations` audit table (`admin_user_id`, `target_user_id`, `badge_id`, `reason`, `revoked_at`) so admins can review without grepping logs.
5. Emit a structured pino line (`event=badge.revoke adminUserId=... targetUserId=... badgeId=... reason=... revocationCount=... repeat=...`).
6. Best-effort `sendBadgeRevoked` email with HTML-escaped reason (dry-run fallback when `RESEND_API_KEY` is unset).
7. `computeBadgeProgress` now returns `REVOKED` (not `LOCKED`) with `revokedAt`, `revokedReason`, and `revocationCount` so the user's `/quests` page can render the in-app notice.

Appeal — user re-submits via the existing AISE / AISE_BUILD claim endpoints. The card on [/quests](#) switches to an "APPEAL" panel showing the admin's reason + repeat counter. On successful re-claim the revocation fields are [cleared](#) and the badge returns to [CLAIMED](#).

Restore — [POST /api/admin/badges/restore](#)
[requireAuth](#) + [requireAdmin](#)): (

1. Validate body: { [userId](#), [badgeId](#), [note?: 1..1000](#) }.
2. 409 if the badge is not currently [REVOKED](#).
3. Clear [revokedAt](#) / [revokedReason](#) / [revokedByUserId](#); set [restoredAt](#) / [restoredByUserId](#) / [restoredNote](#); flip status to [CLAIMED](#). [revoke_history](#) is preserved verbatim.
4. Best-effort [sendBadgeRestored](#) email.
5. [BadgeProgress](#) now also exposes [restoredAt](#) and [restoredNote](#) so the user sees an emerald "restored" panel on [/quests](#).

Admin console — [/admin/badges](#) (admin-only route, TopNav link shown only for admins):

- Revoke form (target UUID, badge select, required reason) that opens a preview dialog showing badge row status, prior revocation count, last reason, the trimmed reason about to be persisted, and a collapsible full revocation timeline ([history\[\]](#), reverse-chronological).
- Repeat-revoke (count "e 2) requires an explicit "I know this is a repeat" acknowledgement checkbox; the confirm button relabels to [CONFIRM Nth REVOKE](#) so it is impossible to miss.
- [GET /api/admin/badges/revocations](#) (limit 1..200) feeds a "Recent Revocations" list with admin + target identities resolved via two aliased joins on [users](#).
- Each row has a one-click [RESTORE](#) button that opens a small dialog with an optional note Textarea; restored rows visually dim with a green pill and a disabled button until the next revoke for the same pair.

Scope is intentionally limited to the **evidence-trust** badges (AISE and AISE_BUILD). Live-computed badges (ASPE, AISA, AISA_PWDD) cannot be meaningfully revoked because they recompute from artifact counts.

3.7 Repository topology (post-update)

```

artifacts-monorepo/
% % % artifacts/
% % % % api-server/
% % % % % src/app.ts      Middleware order, raw-body Stripe mount
% % % % % src/lib/
% % % % % auth.ts         Clerk !' local user JIT bridge
% % % % % tier.ts          Tier + rate-limit gates
% % % % % badges.ts        Live + stored badge logic, SSRF guard
% % % % % cartridge-context.ts  Scope-protected context injection
% % % % % cartridge-credits.ts  Atomic claim/release/link
% % % % % ingestion-credits.ts  Atomic claim/release/link
% % % % % pricing.ts         Stripe price !' tier resolver
% % % % % stripe.ts          Lazy Stripe client
% % % % % src/engines/
% % % % % % shared.ts        callLlm / callLlmJson dispatch
% % % % % % prompts.ts       v2-hardened system prompts
% % % % % % f1..f7.ts         One engine per file
% % % % % % f6vdj.ts          IDE recommender side-step
% % % % % % f8codedj.ts       Code DJ scaffolder
% % % % % % de.ts             DE-SPC evolution
% % % % % % src/routes/
% % % % % % % sessions.ts     Sessions CRUD + serializeArtifact
% % % % % % % harness.ts      POST /api/harness/{f1..f7,evolve,f8}
% % % % % % % ingest.ts       Entry-level ingestion + credits
% % % % % % % cartridge.ts     Premium cartridge SKU
% % % % % % % billing.ts       Stripe checkout (subs + credits)
% % % % % % % stripe-webhook.ts Idempotent webhook
% % % % % % % badges.ts        Live badges + Senior Engineer claim
% % % % % % % %               + admin revoke
% % % % % % % me.ts            Profile, badges, account delete
% % % % % % % verify.ts        Public cert verify (no auth)
% % % % % % % pricing.ts       Public /pricing surface
% % % % % % % exemplars.ts     Hand-curated SPC/PDD library
% % % % % % % cron.ts           Daily rate-limit reset (token-gated)
% % % % % % % health.ts        /healthz
% % % % % % % command-centre/  React + Vite portal
% % % % % % % % src/pages/
% % % % % % % % landing.tsx     Public landing
% % % % % % % % pricing.tsx     Public pricing incl. Cartridge card
% % % % % % % % command.tsx     Authenticated portal home
% % % % % % % % sessions.tsx    Session list
% % % % % % % % session-detail.tsx Sequence rail + per-engine workspaces
% % % % % % % % %               + provider selector + per-engine override
% % % % % % % % % session-new.tsx New session
% % % % % % % % % ingest.tsx    Entry-level ingestion wizard
% % % % % % % % % cartridge.tsx Advanced Cartridge wizard
% % % % % % % % % exemplars.tsx Exemplar library w/ Fork-to-session
% % % % % % % % % quests.tsx    Badge wall + Senior section + downloads
% % % % % % % % % verify.tsx    Public cert verification
% % % % % % % % % account.tsx   Profile, delete
% % % % % % % % % billing.tsx   Stripe customer-portal handoff
% % % % % % % % % demo.tsx      Investor / design-partner demo mode
% % % % % % % % % mockup-sandbox/ Internal preview server
% % % % % % % lib/
% % % % % % % % api-spec/openapi.yaml Single source of truth

```

% % % % api-client-react/

Generated React Query hooks

% % % % api-zod/

Generated request validators

% % % db/src/schema/

Drizzle, one file per table:

```
% % % % users.ts
```

% % % % subscribers.ts

```
%      %      % % % harness-sessions.ts      + preferredModelProvider, cartridgeId
```

% % % % harness-feature-state.ts

```
% % % % harness-artifacts.ts + provider, modelId
```

```
%      %      % % % harness-engine-runs.ts      + provider
```

% % % % harness-escalations.ts

% % % % ingestion-documents.ts

% % % % ingestion-credits.ts

% % % % cartridge.ts

package + credits + docs + spcs + links

% % % % command-centre-badges.ts ASPE/AISA/AISE/AISA_PWDD/AISE_BUILD

% % % % context-craft-badges.ts

% % % % pricing-content.ts

% % % % stripe-webhook-events.ts Idempotency log

% % % email/

Resend wrapper + dev fallback

% % % export/

pdftkit + helpers

% % % integrations-anthropic-ai/ Lazy Anthropic client

% % % integrations-openai-ai/ Lazy OpenAI client

% % % integrations-gemini-ai/ Lazy Gemini client

% % % scripts/

% % % src/md-to-pdf.mjs

Markdown !' PDF (this document)

% % % src/build-atanda-command-centre-pdd-2.mjs Hand-styled investor PDD

% % % src/backfill-artifact-provider.ts

Historical provider backfill

% % % docs/

Investor-ready PDFs (this file lives here)

% % % replit.md

Canonical product + architecture README

3.8 Contracts and guarantees — the load-bearing invariants

- **Contract-first.** Every endpoint declared in `lib/api-spec/openapi.yaml` first. Server validates with generated Zod. Client uses generated TanStack Query hooks. Zero drift.
- **App-layer authorisation, no RLS.** Every Drizzle query that touches user-owned data filters by `req.localUser.id`.
- **Idempotent Stripe webhook.** PK on `event_id` in `stripe_webhook_events`; replays return `{ok:true, replay:true}`; handler exceptions roll back the idempotency row so Stripe can retry.
- **External-first account deletion.** Stripe != Clerk != local cascade != goodbye email. Order is load-bearing.
- **Strict JIT auth sync.** First insert requires a successful Clerk `getUser`; Clerk 404 != 401, Clerk other != 503. Existing users short-circuit so Clerk outages don't lock provisioned accounts.
- **Telemetry is best-effort.** A telemetry-write failure logs a warning and returns; it never breaks an engine response.
- **One canonical SSRF guard.** `lib/badges.ts#headOk` is the only network-side URL verifier. Reused by both AISE and the new Senior Engineer badge. Loosening it requires deliberate review.
- **Cartridge scope is protected.** The PROJECT SCOPE block is always first in the CARTRIDGE CONTEXT and is exempted from the 24 KB cap.
- **F8 refuses uncertified inputs.** Any source MVP PDD without a `spartanCert` returns 400 before any LLM call.
- **No customer API keys.** Every provider integration uses the Replit AI Integrations proxy (`AI_INTEGRATIONS_*` env vars only).

PART 4 — IMPLEMENTATION

The execution sheet. The shipped surface, the open work, the next investments.

4.1 Shipped this cycle (merged tasks, in order)

Task	Title	One-line summary
#1	LLM-agnostic provider dropdown	<code>`callLlm` / `callLlmJson`</code> dispatch over Claude / OpenAI / Gemini; session-level default + per-engine override; tier gate (Explorer locked to Claude); typed errors for unconfigured providers.
#2	Advanced Cartridge Ingestion (\$499.99 / project)	New schema (<code>`cartridge_packages`</code> + <code>`cartridge_credits`</code> + child docs/spcs/links); scope-required wizard; scope-protected context injection on every engine call; Stripe checkout + idempotent webhook branch; F1–F7 auto-unlock on session start.
#3	Senior Architect + Senior Engineer badges	Two new stored ids (<code>`AISA_PWDD`</code> live, <code>`AISE_BUILD`</code> stored); SSRF-hardened URL claim; new Senior section on <code>`quests`</code> .

#4	Shareable Senior badge certificates	PNG / JPEG / SVG export with gold "SENIOR · ADVANCED SYSTEMS" ribbon; per-badge metadata (threshold note, verified URL, evidence excerpt).
#5	Admin Senior Engineer badge revocation	<code>`POST /admin/badges/revoke`</code> ; structured <code>`event=badge.revoke`</code> audit log.
#6	Per-artifact provider + model attribution	Nullable <code>`provider`</code> + <code>`model_id`</code> on <code>`harness_artifacts`</code> ; surfaced on artifact tray with a "Claude · claude-sonnet-4-6"-style badge.
#7	Per-engine model override	Inline dropdown on every workspace; defaults to session preference; single-run override does not mutate session default.
#8	Provider switching verified end-to-end	First api-server vitest suite; per-provider F1 smoke test; lazy Anthropic client; clean skip on unconfigured providers.
#9	Admin revocation audit log + console	New <code>`badge_revocations`</code> table; <code>`GET /admin/badges/revocations`</code> with admin/target identity joins; <code>`/admin/badges`</code> admin page.
#10	User appeal flow for revoked badges	Soft-delete with <code>`REVOKED`</code> status + <code>`revoke_history`</code> jsonb; <code>`BadgeProgress`</code> exposes revoke metadata; <code>`/quests`</code> renders inline notice + APPEAL panel; best-effort revoke email.
#13	Provider + model in artifact detail view	New shared <code>`GeneratedBy`</code> component; rendered above the artifact body in every F1–F8 workspace, with silent fallback for legacy artifacts.
#14	Historical provider backfill	Idempotent script joins <code>`harness_artifacts`</code> to <code>`harness_engine_runs`</code> (most-recent matching run by <code>`session_id`</code> + <code>`engine_id`</code> "d created_at") and populates older NULL provider / <code>`model_id`</code> rows.
#15	Gemini provider fix	One-line <code>`httpOptions.apiVersion: ""`</code> in <code>`@google/genai`</code> client so the Replit Gemini proxy resolves correctly; full 3-provider test matrix now passes.
#16	Provider regressions caught in CI	Root <code>`pnpm run test`</code> + registered validation steps so the per-provider smoke runs pre-merge.

#18	Admin restore endpoint + email + notice	`POST /admin/badges/restore`; clears revoke fields, sets `restoredAt`/`restoredNote`/`restoredByUserId`; preserves `revoke_history`; in-app emerald notice; `sendBadgeRestored` email.
#19	Repeat-revoke warning in admin UI	`GET /admin/badges/revocation-preview` feeds an AlertDialog with prior count, last reason, and an explicit ack checkbox when count "e 2; confirm button relabels to `CONFIRM Nth REVOKE`.
#20	Provider + model on public verify page	`VerifyResult` schema gains `provider` + `modelId`; `verify?cert=...` renders ` <generatedby >`;="" end-to-end="" get="" provenance.<="" recipients="" td=""></generatedby>
#21	Full run telemetry in artifact tray tooltip	`HarnessArtifact` gains `runDurationMs` / `runInputTokens` / `runOutputTokens` / `runAt`; loader joins by sessionId, provider, and modelId; Radix tooltip on the provider chip surfaces the full run.
#24	One-click admin restore button	`RESTORE` button on every revocation row in the admin console; small confirmation dialog with optional note; restored rows dim with a green pill.
#26	Full revoke history in confirm dialog	Collapsible reverse-chronological timeline of all prior revocations inside the revoke AlertDialog; scroll-contained for long histories.
#28	Trimmed reason shown in revoke confirm	The confirm dialog now previews the exact reason about to be persisted and emailed (`data-testid=preview-reason`).

4.2 Verified surface

- `pnpm run typecheck` clean across all packages.
- `pnpm run test` green — per-provider F1 smoke runs against Claude, OpenAI, and Gemini all pass (no skips after the Gemini fix).
- `GET /api/healthz` returns 200; the three workflows (`api-server`, `command-centre`, `mockup-sandbox`) all run.
- Stripe webhook test replay path returns `{ok:true, replay:true}`.
- Senior Engineer claim with a private-network URL is rejected by the SSRF guard.
- Backfill script idempotent (second run reports 0 updates after a full first pass).
- Revoke !' appeal !' restore round-trip exercised against the dev DB with the new admin console.

4.3 In flight (proposed / pending tasks not yet merged)

Task	Title	Investor relevance
------	-------	--------------------

#11	One-click sharing of senior certificates	LinkedIn / portfolio-ready share targets on top of the existing downloads.
#12	Public verification of senior certificates	Stranger-facing verify endpoint for shared certs.
#17	Surface badge revocations to the affected user (notification surface)	Closes the silent-loss gap on revoked badges with an explicit notification.
#22	Show how much each session has cost to run	Surfaces the per-call provider/ model cost telemetry as a per-session total.
#23	Link each artifact directly to the run that produced it	Click-through from artifact to its `harness_engine_runs` row.
#25	Extend the provider-switching matrix beyond F1	End-to-end coverage of F2–F7, F6-VDJ, F8, and DE-SPC across all providers.
#27	Let users pick which Gemini model their sessions run on	Multi-model Gemini selector once #15 is fully battle-tested.

4.4 Roadmap — the next three investments

1. **Cartridge orchestrator service.** Today F1 !' F7 are unlocked one-shot on cartridge session creation and driven by the operator. The plan anticipates a server-side orchestrator that runs the sequence automatically (with rate-limit, tier, and escalation semantics preserved). Lifts the cartridge from "premium ingestion" to "premium turnkey pipeline".
2. **Cartridge !' F8 bundle handoff**The cartridge already produces a PWDD / MVP PDD with full scope anchoring. Wire the certified output into F8 Code DJ as a one-click step from the cartridge completion screen, with a complimentary F8 credit included with the \$499.99 purchase to be decided during product review.
3. **Public certificate gallery.** Operators link to </verify?certId=...> in pitch decks. Build a public, searchable, filterable gallery of issued certificates (operator-opt-in) to turn the verification surface into a true network effect.

4.5 Operational posture summary

Concern	Status
Test mode billing wired	' Stripe test mode; price ids in env
Production deploy	' Replit; one-click; TLS handled
Custom domain	' Supported
Customer-supplied AI keys	' Never accepted
Per-engine cost telemetry	' `harness_engine_runs` includes provider, modelId, tokens, cost, duration
Error reporting	' Sentry on API (`@sentry/node`) and web (`@sentry/react`), gated on DSN env
Daily limits	' Cron-reset via `/api/cron/reset-harness-limits` (token-gated)

Admin actions audit	' Structured pino log for badge revoke
GDPR-style delete	' External-first account delete with cascade
SSRF on operator URLs	' One `headOk` helper, reused everywhere

4.6 What an investor should take away from PWDD v2

The v1 PWDD described a finished product. v2 describes a product that has now also become:

- **Model-portable.** The "you must use Claude" objection is gone. Per- engine choice gives operators an A/B surface and gives us per- provider unit economics.
- **Enterprise-credible.** The \$499.99 Advanced Cartridge SKU consumes a customer's existing knowledge library, their existing SPCs, and a link to their codebase or database — anchored to a project scope they declare up front and that we protect through every downstream engine.
- **Reputationally tiered.** The badge wall now has a Senior section, with both auto-awarded (Architect, three PWDDs) and evidence-claimed (Engineer, verified live URL) recognition, downloadable as shareable certificates, and revocable by admins with a structured audit log.
- **Auditable per artifact.** Every certified output carries the provider and model id that produced it; older outputs have been retroactively populated from telemetry.

These are the changes a serious buyer asks for in week two of a pilot. They are already shipped.

This ATLAS PWDD was produced by applying the F6 ATLAS Drafter shape to the codebase-of-record snapshot at the latest merged checkpoint, then rendered to PDF via the same pdftk + marked pipeline used for the investor PDD library. The underlying technical assertions are auto-synchronised with `replit.md` — the file that travels with the repository.